

# Lab 09 Policy and security posture

AI for IT Security Professionals | TGS-2023039344 | v6.0

## Purpose and output

Create a policy finding register with accountable owners. This lab supports LO1. It uses a simplified deterministic teaching model, not a trained AI system or full Azure emulator. The AI review is a separate exercise using the included prompt.

## Requirements

Python 3.10 or later, a text editor and this entire folder. No packages, network access or API key are required for the baseline. On Windows use `py -3` if `python3` is unavailable. An approved AI tool is optional; the trainer can provide a review response for critique. Azure exercises require the trainer-assigned subscription, permission and feature entitlement. Record an exact blocker where unavailable; never describe an unperformed test as passed.

## Folder contents

- `mock-data.json`: synthetic input with no real personal data or credentials.
- `analyse.py`: runnable analysis for this lab only.
- `expected-results.json`: expected baseline and source checksum.
- `ai-review-prompt.txt`: evidence-constrained AI review prompt.
- `evidence-template.md`: your deliverable template.
- `INSTRUCTIONS.md` and `INSTRUCTIONS.pdf`: the same procedure in two formats.

## 1 Prepare a working copy

Copy this whole folder to a location you can edit. Open a terminal in the copied folder. Confirm Python and inspect the data before running code.

```
python3 --version
python3 -m json.tool mock-data.json
```

Identify the record IDs and explain the meaning of each field. All values are invented classroom fixtures. Open `analyse.py` in the editor and locate the decision rule. It reads a local JSON file and writes a local report.

## 2 Run the baseline

```
python3 analyse.py --input mock-data.json --output results.json
```

Open `results.json`. The expected decisions in output order are: `AUDIT_FINDING`, `WOULD_DENY_REQUEST`, `EXPIRED_EXCEPTION`. The `source_sha256` binds the report to the exact input bytes. Compare against the supplied baseline:

```
python3 -c "import json; a=json.load(open('results.json')); b=json.load(open('expected-results.json')); assert a==b; print('BASELINE PASS')"
```

A pass proves this fixture was processed as specified; it does not prove an Azure control was deployed. Copy the input checksum and decision rows into your evidence template.

## 3 Trace the mechanism

## ***Policy effect semantics***

Audit records noncompliance; deny prevents supported noncompliant resource requests.

Contract: effect, assignment\_scope, resource\_type

Accepted case: Audit flags public exposure for review.

Counterexample: An audit-only assignment is described as blocking deployment.

Diagnosis: Choose the intended effect, examine exemptions and re-evaluate actual resource state.

Evidence: Capture effect, scope, exemption and compliance timestamp.

## ***Posture prioritisation***

A posture recommendation becomes actionable when tied to affected assets and validated exposure.

Contract: recommendation\_id, resource\_id, severity, owner

Accepted case: A critical recommendation affects the internet-facing payment VM.

Counterexample: A stale recommendation references a removed resource.

Diagnosis: Verify existence and exposure before scheduling remediation; score changes are not risk proof.

Evidence: Record resource state, owner, remediation ticket and retest.

## ***Exception lifecycle***

A risk exception is bounded by an owner, compensating control and expiry.

Contract: exception\_id, expires\_utc, owner, compensating\_control

Accepted case: A temporary exception expires after a planned migration.

Counterexample: An indefinite exception suppresses a recurring critical finding.

Diagnosis: Escalate overdue exceptions and restore normal evaluation after expiry.

Evidence: List expired exceptions separately from compliant controls.

## **4 Change one input and test the consequence**

Save a copy of mock-data.json as changed-data.json using the editor. Change pol-a effect to deny. Explain that the model describes a new request, not automatic remediation of an existing resource.

```
python3 analyse.py --input changed-data.json --output changed-results.json
```

Compare decisions and source hashes with the baseline. Identify the exact expression in analyse.py responsible for the changed behaviour. Do not overwrite expected-results.json to make a failing baseline pass. Restore the original fixture if you edited it accidentally.

## **5 Review AI claims against evidence**

Open ai-review-prompt.txt. In an organisation-approved AI tool, submit the prompt with the synthetic input and result only. Record the tool/model name, date and prompt version if available. If no approved AI tool is available, manually draft a tentative analyst summary and label it as a human exercise.

For each returned claim, record its cited ID, the actual field value, whether the claim follows, and any correction. Reject conclusions unsupported by the records even when the model is confident. Include one alternative explanation. The AI response is a proposal; it has no authority to approve or execute a security action.

## 6 Verify the corresponding operational control

### *Azure procedure G Policy and Defender posture*

1. Azure Portal > Policy > Definitions: locate the trainer-approved audit definition for the chosen control. Open its rule and identify the effect and evaluated resource fields.
2. Assign it only to rg-itsec-<initials>. Name the assignment clearly and record parameters, exclusions and any managed identity requirement. Avoid a production Deny assignment for this exercise.
3. Open Compliance and record the evaluation time and affected resource. Policy evaluation is asynchronous; if no current result is available, state that it is pending and retain the assignment evidence.
4. Compare Audit with a trainer-demonstrated Deny request. Explain why the latter describes enforcement on a supported request and does not prove existing resources were repaired.
5. Defender for Cloud > Recommendations: select a finding for the allocated resource. Verify the resource still exists and record severity, exposed surface and remediation guidance.
6. Review secure score, regulatory-compliance view and enabled plan status. Record plan/licence limitations and avoid presenting a numerical score as proof of compliance.
7. Create a remediation record with owner, priority, required approval and retest. Document any exception's compensating control and expiry.
8. Retest the changed control only after an authorised sandbox change. Remove the lab policy assignment when the trainer confirms cleanup; preserve before/after evidence.

For every screenshot or export, record resource scope, UTC time and expected versus observed result. Redact personal data and secrets. If blocked, record the exact error, missing permission/licence, what could be inspected, and the unverified outcome. Ask the trainer to provide an authorised sandbox demonstration where the assessed ability cannot otherwise be evidenced.

## 7 Submit evidence and clean up

Complete evidence-template.md with baseline, changed result, AI claim review and Azure evidence or clearly labelled limitations. Keep results.json and changed-results.json with your submission. Check that your conclusion distinguishes an observed fact from a hypothesis. Remove only resources or assignments you created with trainer approval; do not delete shared training infrastructure.

## Acceptance checks

- The unchanged fixture produces the exact expected report.
- The changed fixture produces the explained result and a different input hash.
- At least one AI or analyst claim is checked against a specific record and field.
- The report states simulation, Azure verified or blocked for each relevant claim.
- The output is a policy finding register with accountable owners with an owner, evidence and remaining uncertainty.

## Troubleshooting

- Python not found: install the trainer-approved Python distribution or use py -3 on Windows.
- File not found: open the terminal in this lab folder; check the input filename.
- JSON parse error: validate with python3 -m json.tool changed-data.json; use lowercase true and false in JSON.
- Baseline mismatch: restore the supplied fixture and inspect the first differing decision; never edit the expected file.
- Azure denial or missing feature: capture the exact error and scope. A blocker is a limitation, not proof of competence or deployment.

- AI invents evidence: reject the claim, cite the missing ID, and request an evidence-limited revision.

## References

The Learner Guide contains the complete source register and the lecture explanations for this lab. Original examples are adapted from the supplied Azure and AI-security references. Product behaviour should be checked against current Microsoft Learn documentation before a real deployment.